

ATMA: Length-Invariant Language Modeling via Polar Attention and Gated-Delta Compression Memory

Anonymous authors

Paper under double-blind review

Abstract

Modern large language models based on softmax scaled-dot-product attention are constrained by their training sequence length: as the key-value sequence grows, softmax probability mass can dilute across a wider distribution, inducing activation shift and long-context performance collapse. Moreover, long-context language modeling faces a structural tension: a sliding-window attention core maintains a bounded local representation and low perplexity but is blind to long-range dependencies, while full-context attention preserves global recall but suffers from out-of-distribution perplexity explosion. To resolve these limitations, we introduce ATMA, a hybrid convolutional-attention architecture that integrates a novel three-channel attention mechanism. ATMA factorizes the attention mixing step into: (1) a count-blind, unit-vector direction channel, (2) a bounded magnitude channel driven by the participation ratio of effective matches over an extreme-value-corrected null sink, and (3) a long-term recurrent compression memory optimized via a gated-delta fast-weights rule. Neither the Polar Attention core nor the recurrent memory is sufficient alone; their combination enables monotonic perplexity reduction and high-fidelity long-range retrieval simultaneously. We evaluate ATMA using a 120-run factorial ablation sweep, demonstrating that the combined Polar + memory model maintains induction needle-in-a-haystack retrieval accuracy above 90% out to 64K tokens ($32\times$ the training length of 2K) while its document perplexity improves monotonically, outperforming softmax-based memory baselines which collapse at extreme context lengths. Code: <https://anonymous.4open.science/r/atma-B798/>.

1 Introduction

The capacity of Large Language Models (LLMs) to ingest, process, and reason over massive context windows is a cornerstone of modern artificial intelligence capabilities. Applications ranging from repository-level code synthesis to complex academic document analysis depend on the model’s ability to maintain coherent representations across tens of thousands of tokens. However, the standard sequence mixer, softmax scaled-dot-product attention (SDPA) (Vaswani et al., 2017), exhibits severe limitations when generalizing to sequence lengths beyond those encountered during training.

This failure of length extrapolation is consistent with a growing body of evidence on softmax dispersion in long contexts. Nakanishi (Nakanishi, 2025) observes that the maximum coordinate of a softmax vector approaches zero as the vector size increases, flattening attention as the context grows. Velićović et al. (Velićović et al., 2025) prove that softmax-based lookup circuits for sharp decisions disperse as problem size increases, while Barbero et al. (Barbero et al., 2024) relate long-sequence failures to representational collapse and over-squashing in decoder-only Transformers. We use dilution to refer to this dense-normalization effect: as the number of keys N grows, probability mass is spread across a larger population, reducing the weight assigned to any fixed set of relevant keys unless logits sharpen with length. Second, softmax attention entangles magnitude information (“how many items matched the query”) with direction information (“what features were matched”) into a single normalized

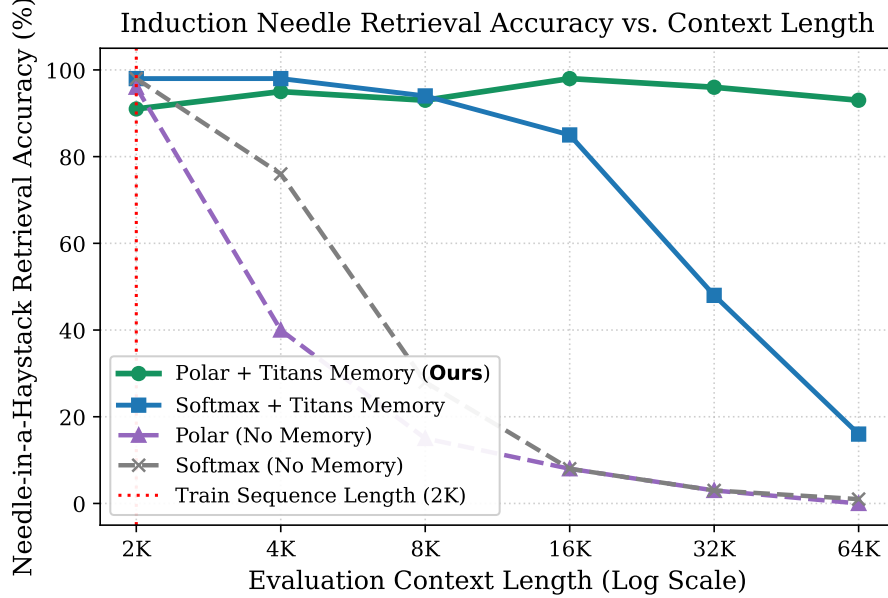


Figure 1: Induction Needle-in-a-Haystack (NIAH) retrieval accuracy comparison under context length extrapolation (up to $32\times$ the training sequence length of 2,048 tokens). Only ATMA (Polar + Titans Memory) holds flat accuracy above 90% out to 64K, whereas the strongest softmax-memory baseline collapses at extreme context lengths.

44 output. When evaluated at longer contexts, this joint distribution can shift the residual
 45 stream away from the regime seen during training.

46 Faced with this challenge, practitioners often employ Sliding Window Attention
 47 (SWA) (Beltagy et al., 2020) or Recurrent/Linear Attention alternatives (Yang et al., 2024b;
 48 Gu & Dao, 2023). However, this introduces a severe trade-off:

- 49 • Sliding Window Attention keeps the active key set bounded and thus maintains
 50 excellent local perplexity, but it is completely blind to any retrieval targets located
 51 past the window boundary.
- 52 • Full Softmax Attention preserves distant recall under specialized conditions, but suf-
 53 fers from severe perplexity collapse as the cumulative attention activations explode
 54 and drift.
- 55 • Recurrent Fast-Weight Memories act as lossy compression systems that carry gen-
 56 eral document flow, but lack the high-precision focus required to recall random,
 57 pinpoint facts (the “needle-in-a-haystack” problem) from a massive context.

We resolve this tension by introducing ATMA, a hybrid sequence-modeling architecture that merges the structural local-mixing advantages of gated depthwise convolutions with a novel three-channel attention layer. In ATMA, each attention layer decomposes its output into an additive combination of three distinct streams:

$$\text{out} = \text{content} + \text{count} + \text{memory} \quad (1)$$

58 The first two streams (content and count) form our proposed Polar Attention core. By
 59 projecting the matched value-sum onto the unit sphere, the content channel isolates what
 60 matched, resulting in a count-blind, size-invariant direction vector. Concurrently, the count
 61 channel isolates how much matched by calculating the participation ratio (inverse Simpson
 62 index) of the attention distribution, bounded via a saturating monotonic map. To prevent
 63 background noise from overwhelming the signal at extreme sequence lengths, Polar Attention
 64 calibrates its logits against a learned, extreme-value-corrected null floor that tracks the
 65 expected maximum of random scores.

The third stream (memory) is a recurrent, per-head associative memory block driven by a gated-delta update rule, inspired by the Titans model (Behrouz et al., 2025). The linear memory behaves as a lossy gist that captures long-term perplexity trends, while Polar Attention supplies the bounded full-context readout needed for exact recall. The ablations show that neither ingredient is enough in isolation: memoryless Polar Attention still loses retrieval at long range, and softmax paired with the same memory collapses at extreme lengths.

To rigorously validate our architectural design, we perform a 120-run factorial ablation sweep, training 370M-parameter models on a 1-billion token FineWeb-Edu corpus, and evaluating them across context lengths from 2K to 64K tokens (up to $32\times$ training length). The results show a clear hierarchy: the combination of Polar Attention and Titans memory resolves the window-vs-retrieval trade-off in our setting. ATMA models hold induction needle-in-a-haystack retrieval flat above 90% across the entire 2K $\rightarrow$ 64K sweep while clean-document perplexity improves monotonically, outperforming both vanilla softmax and softmax-recurrent hybrid baselines.

Our implementation is optimized and numerically cross-verified across three parallel pipelines: a pure-PyTorch reference, an FP16/FP8 training harness, and a paged inference engine. We introduce a FlashAttention-style Triton kernel that handles the streaming participation-ratio calculations in $O(\text{block})$ memory, and we wrap the fast gated-delta training recurrence as opaque custom ops to avoid dynamo compiler graph breaks. This codesign keeps the recurrent memory branch to a 4.5 percentage-point MFU drop on NVIDIA L4 GPUs.

2 Related Work

2.1 Softmax Attention and Scaling Limits

Traditional Transformer architectures rely on softmax scaled-dot-product self-attention (Vaswani et al., 2017). While highly expressive, the $O(T^2)$ computational complexity has inspired numerous long-context extensions. Positional representations like Rotary Position Embeddings (RoPE) (Su et al., 2021) and Attention with Linear Biases (ALiBi) (Press et al., 2021) allow models to extrapolate position indicators to longer sequences. However, they do not directly alter the dense simplex normalization of softmax. Prior work has framed this as flattening, dispersion, or loss of sharpness: Scalable-Softmax rescales logits to counter the vanishing maximum softmax probability (Nakanishi, 2025); adaptive-temperature analyses show that softmax lookup circuits lose sharpness out-of-distribution as size grows (Velickovic et al., 2025); and sparse-attention work proves that softmax distributions become increasingly dispersed whereas entmax can retain nonzero probability on a fixed relevant set (Vasylenko et al., 2026). Sliding Window Attention (SWA) (Beltagy et al., 2020) avoids this dilution by truncating the attention context, but permanently sacrifices long-range recall.

2.2 Linear Attention and Recurrent Fast Weights

To achieve linear-time complexity, linear attention reformulates attention by shifting the computation order of the key, query, and value matrices (Yang et al., 2024b). This reparametrization effectively treats the sequence mixer as a recurrent neural network with a matrix-valued state. Recent models like Gated DeltaNet (Yang et al., 2024b) and Titans (Behrouz et al., 2025) incorporate data-dependent gating and delta-rule updates to actively overwrite and retrieve memories. Titans, in particular, proposes a “Memory-as-Gate” (MAG) or “Memory-as-Layer” approach to store historical context. However, purely linear recurrent states are fundamentally limited by their storage capacity ($O(d_k \cdot d_v)$ parameters); they act as lossy compression systems that struggle to exact-recall rare pinpoint facts (needles) out of massive text haystacks.

2.3 Hybrid Architectures and Canon Layers

Recent work demonstrates that hybrid architectures combining local and global sequence mixers can outperform pure transformers on both efficiency and quality. Models like Mamba (Gu & Dao, 2023) and Liquid Foundation Models 2 (LFM2) (Hasani et al., 2025) combine linear state-space models or gated convolutions with sparse attention layers. Furthermore, Allen-Zhu (2025) show that incorporating depthwise causal convolutions (known as Canon layers) on the query, key, and value projections prior to scoring provides critical horizontal shift-covariance and spatial awareness. ATMA builds on these insights, adopting a 3:1 ratio of LFM2 convolutions to Polar Attention layers to maximize parameter efficiency and sequence throughput.

3 Methodology

3.1 Architecture Overview

ATMA is a 16-layer decoder-only language model designed with a 3:1 hybrid sequence-mixing ratio. Specifically, the architecture consists of 12 LFM2 gated convolutional layers and 4 Polar Attention layers. Each decoder block uses a pre-normalization layout:

$$x' = x + \text{SubLayer}(\text{RMSNorm}(x)) \quad (2)$$

$$x'' = x' + \text{MLP}(\text{RMSNorm}(x')) \quad (3)$$

where the SubLayer is either an LFM2 gated convolution or a Polar Attention layer. The MLP block uses a squared-ReLU activation function with a $4\times$ hidden dimension expansion:

$$\text{MLP}(z) = W_{\text{down}} \left(\text{ReLU}(W_{\text{gate}} z)^2 \cdot (W_{\text{up}} z) \right) \quad (4)$$

By keeping the attention footprint sparse (only 4 out of 16 layers), ATMA maintains low computational overhead while leveraging the global mixing capacity of Polar Attention and Titans memory.

3.2 Polar Attention

Polar Attention is a length-invariant replacement for softmax scaled-dot-product attention. It operates within the standard Grouped-Query Attention (GQA) framework, with H query heads and $H_{KV} = H/4$ key-value heads. Prior to scoring, the horizontal residual causal convolutions (Canon layers) are applied to the q , k , and v projections. We also apply RMS-normalization to the queries and keys.

For a given query i attending over keys $j < n_i$ (where $n_i = i + 1$), the raw scores are computed as:

$$\sigma_{ij} = \frac{q_i \cdot k_j}{\sqrt{d_k}} \quad (5)$$

3.2.1 Length Temperature and Extreme-Value-Corrected Null Floor

To prevent softmax dilution and resist extreme-value noise, we introduce per-head learned scalars that compute two sequence-length-aware quantities:

1. Length Temperature temp_i : Sharpens the attention distribution as context grows, preventing the probability mass from spreading too thin:

$$\text{temp}_i = 1 + \text{softplus}(\alpha) \cdot \log(n_i) \quad (6)$$

where α is a learned head-specific scalar parameter (initialized raw to -1.0).

2. Extreme-Value-Corrected Null Floor null_i : Acts as the logit of a virtual “null sink” key. Because the maximum of n random noise scores grows asymptotically like

$\sqrt{2 \ln n}$, any fixed threshold is eventually overtaken by noise. We correct this by defining a growing floor:

$$\text{null}_i = \text{null_base} + \text{softplus}(\gamma) \cdot \sqrt{\log(n_i + 1)} \quad (7)$$

140 where `null_base` (init 2.0) and γ (init raw 0.5) are learned per-head parameters.

141 3.2.2 Direction Channel (“What”)

We append the null logit null_i to the real key-logits, apply softmax, and form a convex combination of the real values and a learned default null vector $v_{\text{null}} \in \mathbb{R}^{d_k}$:

$$w_i = \text{softmax}([\text{temp}_i \cdot \sigma_{i\bullet}, \text{temp}_i \cdot \text{null}_i]) \quad (8)$$

$$s_i = \sum_j w_{ij} v_j + w_{iN} v_{\text{null}} \quad (9)$$

We then project s_i onto the unit sphere to isolate the direction channel c_i :

$$c_i = \frac{s_i}{\|s_i\|} \quad (10)$$

142 The unit projection ensures that c_i is completely count-blind and size-invariant; it represents
143 purely what feature was attended to, regardless of how many instances matched or the
144 sequence length.

145 3.2.3 Magnitude Channel (“How Much”)

To represent how many effective matches were found, we reuse the attention weights w_i . We first renormalize them over the real keys only, and then calculate the participation ratio n_{eff} (which corresponds to the inverse Simpson index):

$$\hat{w}_{ij} = \frac{w_{ij}}{\sum_k w_{ik}} \quad (11)$$

$$n_{\text{eff}} = \frac{1}{\sum_j \hat{w}_{ij}^2} \quad (12)$$

If 1,000 strong keys match, $n_{\text{eff}} \approx 1000$; if only noise is present, the length-aware temperature keeps n_{eff} bounded. We gate this count by the confidence that real signal was found, $m_{\text{eff}} = n_{\text{eff}}(1 - w_{iN})$, and map it through a bounded, saturating monotonic function:

$$\text{mag}_i = \tanh(\text{softplus}(\beta) \cdot \log(1 + m_{\text{eff}})) \in [0, 1] \quad (13)$$

146 This bounded map ensures that the magnitude channel input remains in-distribution at any
147 context length (unlike raw $\log(m)$ which grows without bound).

148 3.2.4 Assembly and Distractor Objective

The direction and magnitude are recombined into the residual stream:

$$\text{out}_{\text{polar}} = W_o(\text{reshape}(c) \cdot \sigma(\text{gate})) + W_\mu(\text{mag}) \quad (14)$$

149 where `gate` is a sigmoid gating factor carried in the q projection, W_o is the output projection,
150 and W_μ is a per-head additive projection.

To calibrate the null floor during training, we introduce an auxiliary distractor loss $\mathcal{L}_{\text{align}}$. A set of R random keys are projected and scored, and they must lose to the null sink:

$$\mathcal{L}_{\text{align}} = \text{mean softmax}([\text{temp} \cdot \sigma_{\text{rand}}, \text{temp} \cdot \text{null}]) \Big|_{\text{rand}} \quad (15)$$

151 This loss (weighted by 0.01 when enabled) pushes random distractors below the null floor,
152 widening the signal-to-noise margin Δ . In the final ablation, however, the best Polar +
153 memory configuration leaves this auxiliary objective disabled: once the recurrent memory
154 branch is active, the distractor objective over-sharpens the null floor and hurts retrieval
155 (Section 5.3).

156 3.3 Titans Memory-as-Gate Integration

While Polar Attention preserves length-invariant activations, a single attention core cannot solve the window-vs-retrieval trade-off alone. We incorporate a long-term recurrent memory as an additive third channel in the residual block:

$$\mathbf{out} = \mathbf{out}_{\text{polar}} + \mathbf{out}_{\text{mem}} \quad (16)$$

Each head maintains a matrix state $M \in \mathbb{R}^{d_v \times d_k}$ acting as a fast-weight associative key-value store. It reuses the layer’s q , k , and v projections, and derives two data-dependent gates per step from the layer input x :

$$\gamma_t = \sigma(W_\gamma x_t + b_\gamma) \in (0, 1) \quad (\text{retention gate; } b_\gamma \text{ init } 3.9 \rightarrow \sigma \approx 0.98) \quad (17)$$

$$\beta_t = \sigma(W_\beta x_t + b_\beta) \in (0, 1) \quad (\text{write strength; } b_\beta \text{ init } 0.0 \rightarrow \sigma \approx 0.5) \quad (18)$$

157 3.3.1 Gated-Delta Recurrence and Key Normalization

The memory state is updated via a conditional gated-delta rule, which we implement following the Flash Linear Attention (FLA) library (Yang et al., 2024c) canonical convention (decay-first, undecayed write, self-inclusive readout). Crucially, when momentum $\eta = 0$, the Titans neural memory reparametrizes exactly to a closed-loop Gated DeltaNet (GDN) recurrence (Yang et al., 2024a):

$$M_t = \gamma_t \cdot M_{t-1} (I - \beta_t \cdot k_t k_t^\top) + \beta_t \cdot v_t k_t^\top \quad (19)$$

$$r_t = M_t \cdot q_t \quad (20)$$

This can be computed in an equivalent per-step online manner:

$$M \leftarrow \gamma_t M \quad (21)$$

$$\text{pred} = M k_t \quad (22)$$

$$M \leftarrow M + \beta_t (v_t - \text{pred}) k_t^\top \quad (23)$$

$$r_t = M q_t \quad (24)$$

158 Finding 1: Key Normalization. The eigenvalue of the delta rule update along k is $\gamma(1 -$
 159 $\beta \|k\|^2)$. Standard RMS-normalization yields $\|k\|^2 = d_k$, pushing the eigenvalue to ≈ -7 ,
 160 which causes immediate exponential divergence (the state norm explodes to $\approx 10^{57}$). We
 161 resolve this by applying L_2 -normalization to the keys and queries ($\|k\|_2 = 1$). This bounds
 162 the eigenvalue in $(0, 1)$, ensuring absolute stability.

163 Finding 2: Self-Stabilization. Analysis of the recurrent scan reveals that the delta memory
 164 is self-stabilizing. Because the $(I - \beta k k^\top)$ term continuously projects out old matched
 165 dimensions, the state norm remains completely flat across length sweeps even at $\gamma = 1$.
 166 Gamma γ behaves purely as a temporal-horizon dial (governing recency vs global memory),
 167 not a stability requirement.

168 3.3.2 Readout Assembly

The readout vector r_t is normalized and gated before adding it to the residual stream:

$$\mathbf{out}_{\text{mem}} = W_{\text{mem_proj}} (\text{RMSNorm}(r) \cdot \sigma(\text{gate}_{\text{mem}}(x))) \quad (25)$$

169 We initialize $W_{\text{mem_proj}}$ to zero so that the memory branch acts as a safe no-op at step 0 of
 170 training or when fine-tuning a pre-trained checkpoint.

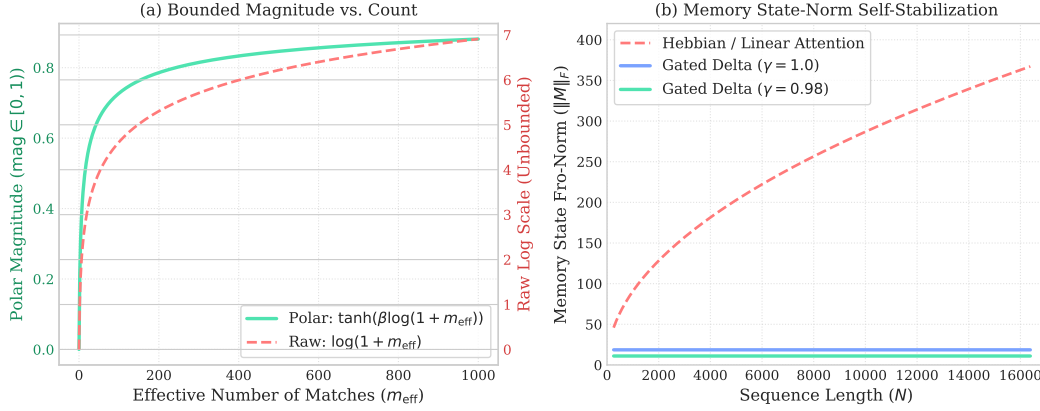


Figure 2: Mechanistic behavior of the ATMA sequence-mixer. (a) Under Polar Attention, the count channel maps the effective number of matches m_{eff} monotonically to a bounded interval $[0, 1]$ using a saturating $\tanh$ function, preventing representation shift. (b) In the Titans memory branch, the gated-delta recurrence self-stabilizes, keeping the state norm $\|M\|_F$ flat across long context sequences, whereas a standard Hebbian/linear-attention memory exhibits square-root growth ($\approx \sqrt{N}$), leading to divergence.

171 4 Experimental Setup

172 To evaluate the length-extrapolation and retrieval capabilities of ATMA, we perform a
173 factorial sweep across 120 completed configurations.

174 4.1 Model Configuration and Training

175 All ablation candidates share a fixed model architecture to isolate the impact of the sequence
176 mixer:

- 177 • Parameters: 369.72M non-embedding parameters.
- 178 • Layers: 16 layers (12 LFM2 gated convolutions, 4 Attention layers).
- 179 • Dimensions: Hidden size $d_{\text{model}} = 1024$, head dimension $d_k = d_v = 128$.
- 180 • Heads: 8 query heads, 2 KV heads (Grouped Query Attention ratio of 1:4).
- 181 • Vocabulary: 50,304 (GPT-2 tokenizer).

182 We train each candidate for exactly 1B tokens (1 epoch) on the FineWeb-Edu corpus, using
183 a sequence length of 2048. We use the Muon optimizer for coordinate-descent weight up-
184 dates on the sequence-mixer and MLP blocks, combined with AdamW on embedding and
185 normalization layers. The learning rate uses a cosine schedule with a 70% cooldown fraction.

186 4.2 Ablation Grid Axes

187 Our 120-run factorial sweep evaluates combinations across five categorical axes:

- 188 1. Attention Type (attn_type): polar (our proposed Polar Attention), rope (standard
189 softmax attention with Rotary Position Embeddings (Su et al., 2021)), and nope
190 (softmax attention without positional encodings, which organically develop posi-
191 tional representations in causal transformers (Haviv et al., 2022; Wang et al., 2024;
192 Zuo et al., 2025), relying on LFM2 convolutions for spatial awareness).
- 193 2. Regularizer Mode (reg_mode): baseline, weak, strong, discrete, and zipfian (varying
194 the strength and layout of signature-stream regularization; see Appendix A).
- 195 3. Distractor Loss (distractor): off vs on (calibrating the null floor with $R = 2048$
196 random keys).

- 197 4. Memory Branch (memory): off vs on (enabling the Gated-Delta Titans memory
198 branch).
199 5. Sliding Window (window): off (full global attention) vs on (sliding window of width
200 1024 on the attention core).

201 4.3 Evaluation Probes

202 Each trained model is evaluated across sequence multipliers from $1\times$ to $32\times$ the training
203 length (2,048 to 65,536 tokens) using two critical probes:

- 204 1. Document Perplexity: Measured on single, coherent long documents from the
205 FinePDFs corpus.
206 2. Induction Needle-in-a-Haystack: We plant an induction needle of the form “The
207 access code for record [KEY] is [D1] [D2] [D3] [D4] [D5]” near the start of a long
208 text haystack, then repeat the record-specific cue after a controlled gap and score
209 greedy per-digit accuracy on the five-digit value at distances up to 64K tokens.

210 5 Experimental Results and Analysis

211 5.1 The Attention Generalization Failure

212 We first analyze the performance of attention-only models (with no memory branch or
213 windowing). Table 1 highlights a stark collapse in both perplexity and retrieval as sequence
214 length exceeds the training threshold.

Length	Clean Perplexity (nats) ↓			Needle Accuracy (%) ↑		
	NoPE	Polar	RoPE	NoPE	Polar	RoPE
2,048 ($1\times$)	2.76	2.83	2.85	98%	96%	43%
8,192 ($4\times$)	2.67	3.32	2.78	28%	15%	4%
32,768 ($16\times$)	3.38	3.60	3.07	3%	3%	0%
65,536 ($32\times$)	3.71	3.60	3.36	1%	0%	0%

Table 1: Performance of memoryless, windowless attention models. Softmax (NoPE), polar, and RoPE attention alone degrade in perplexity and experience retrieval collapse past $4\times$ training length.

215 This baseline diagnosis reveals that attention alone cannot generalize. Softmax dilutes its
216 attention probability as context grows, while the polar core’s unconstrained participation
217 ratio n_{eff} shifts the activation range, pushing downstream layers out-of-distribution.

218 5.2 The Titans Memory Unlock

219 Enabling the recurrent Titans gated-delta memory branch on top of the global Polar At-
220 tention core completely reverses this trend. The results for our winning configuration (full
221 polar + Titans memory, no window, no distractor) are shown in Table 2.

222 The combination of Polar Attention and Titans memory achieves the best of both worlds:

- 223 1. Monotonic Perplexity Reduction: Document perplexity falls consistently from 2.70
224 nats down to 1.96 nats at 64K tokens, improving over the softmax-memory baseline’s
225 2.34 nats and the RoPE-memory’s 2.86 nats at 64K, demonstrating that the model
226 actively uses the longer context to improve predictions.
227 2. Flat Retrieval Generalization: Retrieval accuracy remains flat at 91–98% (length-
228 weighted average of 94%) out to $32\times$ training length.
229 3. Polar vs. Baselines Contrast: While Softmax + Titans memory performs well at
230 shorter context lengths, it collapses to 16% retrieval at 64K. RoPE + Titans memory
231 collapses even faster, falling to 0% retrieval by 32K context. This occurs because

Distance	Needle Accuracy (%) $\uparrow$			Clean Perplexity (nats) $\downarrow$		
	NoPE+Mem	RoPE+Mem	Ours	NoPE+Mem	RoPE+Mem	Ours
2,048 ($1\times$)	98%	74%	91%	2.66	2.81	2.70
4,096 ($2\times$)	98%	55%	95%	2.52	2.72	2.45
8,192 ($4\times$)	94%	29%	93%	2.39	2.74	2.22
16,384 ($8\times$)	85%	9%	98%	2.29	2.79	2.09
32,768 ($16\times$)	48%	0%	96%	2.29	2.82	2.01
65,536 ($32\times$)	16%	0%	93%	2.34	2.86	1.96

Table 2: Retrieval accuracy and document perplexity comparison under long-context extrapolation. Softmax (NoPE) + Titans memory improves perplexity but collapses in retrieval at extreme length; RoPE + Titans memory collapses even faster. Polar + Titans memory holds retrieval above 90% out to 64K and achieves the best 64K perplexity.

neither traditional softmax nor RoPE representations are length-invariant, leading to activation drift that corrupts the memory readout.

5.3 Ablation of Windowing and Distractors

Surprisingly, when the memory branch is active, adding sliding windowing or distractor losses actually hurts retrieval, as shown in Table 3.

Needle Distance	Polar + Mem (Winner)	+ Distractor	+ Window (1024)
2,048 ($1\times$)	91%	74%	51%
16,384 ($8\times$)	98%	76%	53%
65,536 ($32\times$)	93%	59%	30%

Table 3: Impact of sliding-window and distractor loss when the memory is enabled. Windowing makes the attention core blind past width 1024, and the distractor over-corrects the floor, both harming retrieval.

A sliding window restricts the attention layer from training on long-context patterns, making it unable to align its projections for distant keys. The distractor loss over-sharpens the null floor, which suppresses the weak but necessary signals retrieved from the long-term memory. Thus, the simplest configuration (full polar + memory) is strictly superior.

5.4 Open-Weight Pretrained Baselines

The controlled ablation above isolates architecture under a fixed 370M-scale training budget, but it is also useful to anchor the numbers against publicly available pretrained models. We therefore run the same clean-document, junk-stream, and induction-NIAH probes on open-weight baselines ranging from 230M to 752M parameters. Since each pretrained model uses its own tokenizer, language-modeling scores are reported as bits per GPT-2 token; lower is better. For $\mathcal{L} = \{2048, 4096, 8192, 16384, 32768, 65536\}$ and each length-indexed metric x_L , the weighted average is

$$\text{WAvg}(x) = \frac{\sum_{L \in \mathcal{L}} (L/2048) x_L}{\sum_{L \in \mathcal{L}} (L/2048)}, \quad (26)$$

so the 64K endpoint dominates the summary.

To contextualize the comparison, Table 4 reports the public training-token budgets and native/pretraining context windows for these baselines. We estimate training compute using the standard dense language-model approximation $F \approx 6ND$, where N is parameter count and D is training tokens; these FLOP estimates are order-of-magnitude context rather than exact vendor accounting.

Model / Recipe	Params	Train Tokens	Native Ctx	Train FLOPs
Qwen3.5-0.8B-Base	752M	N/A	262K	N/A
Qwen3-0.6B-Base	596M	36T	32K	128.7 ZF
Qwen2.5-0.5B	494M	18T	32K	53.4 ZF
Granite-4.0-H-350M	340M	15T	32K	30.6 ZF
Gemma-3-270M	268M	6T	32K	9.6 ZF
Granite-4.0-350M	352M	15T	32K	31.7 ZF
Falcon-H1-0.5B	521M	2.5T	16K	7.8 ZF
SmolLM2-360M	362M	4T	8K	8.7 ZF
LFM2.5-230M	230M	19T	32K	26.2 ZF
LFM2.5-350M	354M	28T	32K	59.5 ZF
ATMA ablation models	370–378M	1B	2K	0.0022–0.0023 ZF

Table 4: Training-budget context for the open-weight baselines and ATMA ablations. FLOPs are approximate dense LM training FLOPs computed as $6 \times \text{parameters} \times \text{training tokens}$. ZF denotes 10^{21} floating-point operations.

Model / Recipe	Params	Clean			Junk			Needle		
		WAvg	2K	64K	WAvg	2K	64K	WAvg	2K	64K
Qwen3.5-0.8B-Base	752M	1.70	2.23	1.54	3.80	3.75	3.80	100.0	100.0	100.0
Qwen3-0.6B-Base	596M	1.72	2.21	1.60	3.80	3.70	3.84	97.5	100.0	95.0
Qwen2.5-0.5B	494M	1.84	2.30	1.74	3.83	3.75	3.87	68.4	100.0	38.1
Granite-4.0-H-350M	340M	2.20	2.25	2.11	3.95	3.74	3.98	56.9	75.0	54.4
Gemma-3-270M	268M	2.22	2.89	2.18	4.41	4.59	4.54	53.2	96.2	24.4
Granite-4.0-350M	352M	2.69	2.56	3.10	4.75	3.76	5.65	25.6	40.6	8.8
Falcon-H1-0.5B	521M	3.03	2.57	3.05	5.31	3.67	5.65	28.6	92.5	19.4
SmolLM2-360M	362M	5.57	2.43	6.72	8.60	3.58	10.37	16.9	100.0	8.1
LFM2.5-230M	230M	6.93	3.59	6.96	10.55	5.37	11.01	29.0	38.1	19.4
LFM2.5-350M	354M	6.96	3.79	6.90	10.78	5.82	11.11	44.6	45.6	46.3
ATMA Polar + Titans Memory	378M	3.04	3.90	2.83	4.49	4.49	4.51	94.1	91.3	92.5
ATMA Softmax + Titans Memory	378M	3.37	3.84	3.38	4.58	4.46	4.66	41.7	97.5	16.3
ATMA RoPE + Titans Memory	378M	4.08	4.05	4.12	4.98	4.46	5.16	5.9	73.8	0.0
ATMA Polar, no Memory	370M	5.11	4.08	5.20	7.69	4.71	8.19	5.3	96.3	0.0
ATMA Softmax, no Memory	370M	4.93	3.99	5.35	8.57	4.57	9.63	7.9	97.5	1.3
ATMA RoPE, no Memory	370M	4.55	4.12	4.84	6.66	4.46	7.51	1.2	42.5	0.0

Table 5: Open-weight pretrained baselines evaluated with the same probes as ATMA. Clean and junk metrics are bits per GPT-2 token; Needle is greedy per-digit accuracy in percent. The open Qwen baselines are much stronger pretrained language models and solve the synthetic probe, while the controlled ATMA comparison shows that Polar + Titans memory is the only ATMA recipe that keeps retrieval high at 64K.

248 These results narrow, rather than weaken, the claim. ATMA is not yet competitive with
249 the strongest open pretrained models in absolute language-modeling quality; Qwen3.5-0.8B,
250 in particular, has both lower bits-per-token and perfect 64K induction accuracy in this
251 probe. The architectural result is instead that, under the matched ATMA training recipe
252 and roughly 1B-token budget, Polar + Titans memory is the component combination that
253 preserves long-range retrieval, whereas ATMA softmax-memory and memoryless variants
254 collapse at the same 64K endpoint.

255 5.5 Hardware-Software Codesign and Kernel Machinery

256 To ensure that ATMA’s mathematical properties translate into real-world efficiency, we
257 perform a hardware-software codesign. We optimize the sequence mixers by implementing
258 custom GPU kernels using the Triton programming language, integrating them across our
259 training, reference, and paged inference pipelines.

260 5.5.1 Fused Triton Polar Attention Kernel

261 Standard attention implementations fail to handle Polar Attention efficiently due to the
262 $O(T^2)$ memory cost of materializing the participation ratio’s intermediate scores. To

263 achieve $O(\text{block})$ memory in both the forward and backward passes, we implement a fused
264 FlashAttention-style Triton kernel with query- and key-blocking.

To compute the participation ratio online, we must track an extra streamed accumulator $Q^2 = \sum_j \exp(\text{temp} \cdot \sigma_{ij} - M)^2$ alongside the standard running max M and denominator sum $L = \sum_j \exp(\text{temp} \cdot \sigma_{ij} - M)$. On each max update from M_{old} to M_{new} , we define the correction factor $\alpha = \exp(M_{\text{old}} - M_{\text{new}})$. The accumulators are rescaled as follows:

$$L \leftarrow \alpha \cdot L \quad (27)$$

$$S \leftarrow \alpha \cdot S \quad (28)$$

$$Q^2 \leftarrow \alpha^2 \cdot Q^2 \quad (29)$$

265 The squared correction factor α^2 corrects the quadratic term in the participation ratio
266 denominator. This enables exact numerical recovery of the participation ratio $n_{\text{eff}} = L^2/Q^2$
267 at the end of the streaming reduction. The backward pass splits work by running a cheap
268 query preamble in PyTorch and executing the heavy $O(T^2)$ gradient loops ($dq, dk/dv$) as
269 optimized Triton loops, running 7–27 $\times$ faster and using 5 $\times$ less peak memory than the
270 PyTorch eager baseline on an NVIDIA L4 GPU.

271 5.5.2 GQA-Grouped Paged Polar Decode Kernel

272 During the decode phase of inference, standard sequence mixers suffer from significant high-
273 bandwidth memory (HBM) bottlenecks due to gathering paged KV cache buffers. We
274 implement a custom polar_attention_decode kernel that reads directly from paged KV
275 cache blocks using a sequence block table, making it completely CUDA-graph-capturable.

276 To maximize throughput, the kernel is GQA-grouped: a single threadblock serves a sequence
277 and an entire KV head group. The block table loads the cached keys and values once into
278 shared memory, and uses them to compute the scores for all 4 associated query heads
279 in parallel, reducing HBM read traffic by 4 $\times$. Furthermore, the key loop is dynamically
280 bounded by the live context length rather than a graph-padded maximum sequence length,
281 minimizing unnecessary flops during early generation steps.

282 5.5.3 In-Place Recurrent Gated-Delta Step Kernel

283 The recurrent Titans memory state represents a size $H \times d_k \times d_v$ tensor per sequence-layer
284 (approx. 512 KB/seq/layer at FP32), making memory updates the dominant decode bottle-
285 neck at large batch sizes. Standard implementations gather the recurrent state, perform a
286 batched matrix-update kernel, and scatter the state back, incurring massive HBM traffic.

287 To address this, we implement a fused, in-place gated-delta step kernel
288 (kernel/gated_delta_triton.py). During each decode step, the kernel loads the se-
289 quence state, computes the rank-1 gated-delta write $M_t = \gamma_t M_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$
290 in-place, and writes the updated state directly back to the slot-indexed sequence state table.
291 This eliminates the gather-scatter cycle entirely, reducing memory traffic by 3 $\times$.

292 5.5.4 Empirical Systems Performance and Overhead

293 We evaluate our systems optimizations on an NVIDIA L4 GPU (24GB).

- 294 • Training Efficiency (MFU): The gated-delta recurrence contains sequential cross-
295 step dependencies that force PyTorch Dynamo compiler graph breaks. We wrap
296 the Flash Linear Attention forward and backward passes as opaque custom ops
297 (FLA_CUSTOM_OP=1) with fake shape registrations. This allows Dynamo to
298 compile the surrounding layers cleanly and enables backward activation recompu-
299 tation. As shown in Table 6, the recurrent memory branch reduces MFU by 4.5
300 percentage points relative to the Polar-only training run.
- 301 • Inference Throughput: Our GQA-grouped paged decode kernel and in-place gated-
302 delta step kernel achieve a decode throughput of 19,270 tokens/second at a batch
303 size of 512.

Configuration	Avg Step Time	MFU	Relative MFU Drop
Polar	28.33 s	40.1%	–
Polar + Memory	32.49 s	35.6%	11.2%
Polar + Memory + Distractor ($R = 2048$)	36.39 s	31.8%	20.7%

Table 6: Training throughput of the Polar ATMA configurations on NVIDIA L4 GPUs at sequence length 2048. The memory branch reduces MFU by 4.5 percentage points relative to the Polar-only run; adding the optional distractor loss incurs additional overhead and is not used in the winning configuration.

6 Conclusion

We presented ATMA, a hybrid sequence mixer that resolves the long-context perplexity-retrieval trade-off. By pairing a length-invariant Polar Attention core with a recurrent gated-delta Titans memory, ATMA retains flat induction needle retrieval accuracy above 90% out to 64K tokens ($32\times$ training length) while document perplexity reduces monotonically. We rigorously verified our design through a 120-run sweep, and demonstrated that hardware-level software codesign keeps the memory branch to a 4.5 percentage-point MFU drop on NVIDIA L4 GPUs. Future work will investigate extending write-path distractors to further enhance memory capacity.

References

- Habibullah Akbar. Weak-sigreg: Covariance regularization for stable deep learning. arXiv preprint arXiv:2603.05924, 2026.
- Zeyuan Allen-Zhu. Physics of language models: Part 4.1, architecture design and the magic of canon layers. arXiv preprint arXiv:2512.17351, 2025.
- Randall Balestriero and Yann LeCun. Lejepa: Provable and scalable self-supervised learning without the heuristics. arXiv preprint arXiv:2511.08544, 2025.
- Federico Barbero, Andrea Banino, Steven Kapturowski, Dharshan Kumaran, Joao G. M. Araujo, Alex Vitvitskyi, Razvan Pascanu, and Petar Velickovic. Transformers need glasses! information over-squashing in language tasks. In Advances in Neural Information Processing Systems, 2024.
- Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. arXiv preprint arXiv:2501.00663, 2025.
- Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. arXiv preprint arXiv:2004.05150, 2020.
- Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. arXiv preprint arXiv:2312.00752, 2023.
- Ramin Hasani et al. Lfm2 technical report. arXiv preprint arXiv:2511.23404, 2025.
- Adi Haviv, Ori Ram, Ofir Press, Peter Izsak, and Omer Levy. Transformer language models without positional encodings still learn positional information. arXiv preprint arXiv:2203.16634, 2022.
- Ken M. Nakanishi. Scalable-softmax is superior for attention. arXiv preprint arXiv:2501.19399, 2025.
- Ofir Press, Noah A Smith, and Mike Lewis. Train short, test long: Attention with linear biases enables input length extrapolation. arXiv preprint arXiv:2108.12409, 2021.
- Jianlin Su et al. Roformer: Enhanced transformer with rotary position embedding. arXiv preprint arXiv:2104.09864, 2021.

- 340 Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez,
341 Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural*
342 *Information Processing Systems*, volume 30, pp. 5998–6008. Curran Associates, Inc., 2017.
- 343 Pavlo Vasylenko, Hugo Pitorro, Andre F. T. Martins, and Marcos Treviso. Long-context
344 generalization with sparse attention. In *International Conference on Learning Represen-*
345 *tations*, 2026.
- 346 Petar Velickovic, Christos Perivolaropoulos, Federico Barbero, and Razvan Pascanu. Soft-
347 max is not enough (for sharp size generalisation). In *International Conference on Machine*
348 *Learning*, 2025.
- 349 Jie Wang, Tao Ji, Yuanbin Wu, Hang Yan, Tao Gui, Qi Zhang, Xuanjing Huang, and
350 Xiaoling Wang. Length generalization of causal transformers without position encoding.
351 arXiv preprint arXiv:2404.12224, 2024.
- 352 Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2
353 with delta rule. arXiv preprint arXiv:2412.06464, 2024a.
- 354 Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear
355 transformers with the delta rule over sequence length. arXiv preprint arXiv:2406.06484,
356 2024b.
- 357 Songlin Yang et al. Flash linear attention. [https://github.com/fla-org/](https://github.com/fla-org/flash-linear-attention)
358 [flash-linear-attention](https://github.com/fla-org/flash-linear-attention), 2024c.
- 359 Chunsheng Zuo, Pavel Guerzhoy, and Michael Guerzhoy. Position information emerges in
360 causal transformers without positional encodings via similarity of nearby embeddings.
361 In *Proceedings of the 31st International Conference on Computational Linguistics*, pp.
362 9418–9430, Abu Dhabi, UAE, jan 2025. Association for Computational Linguistics. URL
363 <https://aclanthology.org/2025.coling-main.632/>.

A Regularization Modes

The ablation grid varies a representation regularizer applied to the model’s signature stream. For all non-baseline settings we use the same sweep weight, $\alpha_{\text{sig}} = 0.01$, and train with

$$\mathcal{L} = (1 - \alpha_{\text{sig}})\mathcal{L}_{\text{LM}} + \alpha_{\text{sig}}\mathcal{L}_{\text{reg}} + \lambda_{\text{dist}}\mathcal{L}_{\text{dist}}. \quad (30)$$

The weak and strong modes follow the two SIGReg families introduced by Weak-SIGReg (Akbar, 2026) and LeJEPA’s strong SIGReg objective (Balestrieri & LeCun, 2025); the discrete and zipfian modes are local variants that interpolate those ideas with stronger geometric priors.

Mode	Meaning in the sweep
baseline	No signature-stream regularization is used; equivalently, $\alpha_{\text{sig}} = 0$.
weak	Weak SIGReg: center a random sketch of the token representations and match its covariance to the identity, $\ \text{Cov}(Sx) - I\ _F$. This encourages decorrelated, unit-variance features at low cost.
strong	Strong SIGReg: project representations onto random one-dimensional directions and match their empirical characteristic functions to the standard Gaussian characteristic function over a fixed grid of frequencies.
discrete	Local variant: first normalize each representation to the sphere of radius $\sqrt{d}$, then apply the weak covariance-matching objective. This preserves the whitening pressure while encouraging fixed-norm, discrete-code-like directions.
zipfian	Local variant: normalize directions for an orthogonality penalty, then match the sorted representation magnitudes to a Zipf profile r^{-1} . This biases the signature stream toward heavy-tailed feature usage rather than uniform energy allocation.

Table 7: Regularization modes used in the ablation grid. The weak and strong modes are taken from SIGReg-style covariance and distribution matching; the discrete and zipfian modes are improvised variants used to probe stronger geometric structure.

B Polar Attention Algorithms

This appendix gives code-level implementations for Polar Attention. Code 1 shows the materialized PyTorch oracle (polar_reduce in model/blocks.py), which is used for numerical verification. Code 2 shows the core of the FlashAttention-style Triton forward kernel (kernel/polar_triton.py), which streams keys in blocks and maintains the same sufficient statistics without materializing the $T \times T$ score matrix.

375 B.1 Naive Materialized PyTorch Reference

Listing 1: Materialized PyTorch reference for Polar Attention.

```

376 def polar_reduce(sigma, v, n_keys, *, v_null, null_base,
377                 null_slope_raw, len_gain_raw, mag_beta_raw,
378                 eps=1e-6):
379     """Reference oracle. sigma: (B,H,Tq,Tk), v: (B,H,Tk,dk)."""
380     out_dtype = v.dtype
381     cd = torch.float32 if v.dtype in (torch.float16, torch.bfloat16) else v.dtype
382     B, H, Tq, Tk = sigma.shape
383     dk = v.shape[-1]
384     sigma, v = sigma.to(cd), v.to(cd)
385
386     n = n_keys.to(cd).clamp(min=1.0)
387     temp = 1.0 + F.softplus(len_gain_raw.to(cd)).view(1, H, 1, 1) \
388         * torch.log(n).view(1, 1, Tq, 1)
389     null = null_base.to(cd).view(1, H, 1, 1) \
390         + F.softplus(null_slope_raw.to(cd)).view(1, H, 1, 1) \
391         * torch.sqrt(torch.log(n + 1.0)).view(1, 1, Tq, 1)
392
393     # Scale finite entries, then re-apply the mask. This avoids NaNs from
394     # differentiating through temp * (-inf).
395     masked = torch.isneginf(sigma)
396     sigma_safe = torch.where(masked, torch.zeros_like(sigma), sigma)
397     real = (sigma_safe * temp).masked_fill(masked, float("-inf"))
398
399     logits = torch.cat([real, null.expand(B, H, Tq, 1) * temp], dim=-1)
400     w = torch.softmax(logits, dim=-1)
401     w_r = w[..., :-1]
402     w_null = w[..., -1:]
403
404     # Direction channel: convex value mix plus learned null vector, projected
405     # to the unit sphere so the channel is count-blind.
406     s = torch.matmul(w_r, v) + w_null * v_null.to(cd).view(1, H, 1, dk)
407     c = F.normalize(s, p=2, dim=-1, eps=eps)
408
409     # Magnitude channel: inverse Simpson participation ratio over real keys,
410     # gated by non-null confidence and squashed to [0, 1).
411     denom = w_r.sum(-1, keepdim=True).clamp_min(eps)
412     w_hat = w_r / denom
413     n_eff = 1.0 / w_hat.square().sum(-1).clamp_min(eps)
414     m_eff = n_eff * (1.0 - w_null.squeeze(-1))
415     beta = F.softplus(mag_beta_raw.to(cd)).view(1, H, 1)
416     mag = torch.tanh(beta * torch.log1p(m_eff))
417
418     return c.to(out_dtype), mag.to(out_dtype)
419
420

```


B.2 Flash-Style Triton Forward Kernel

Listing 2: Core Triton streaming forward pass. The full kernel includes pointer arithmetic, launch metadata, and backward kernels.

```

422 @triton.jit
423 def _polar_fwd_kernel_core(q, K, V, VNULL, SPG, NULLBASE, SPS, BETA,
424                          n_i, h, Tk, scale, eps,
425                          BLOCK_N: tl.constexpr, DK: tl.constexpr,
426                          WINDOW: tl.constexpr):
427     # Per-query length temperature and extreme-value-corrected null floor.
428     if WINDOW > 0:
429         n_count = tl.maximum(tl.minimum(n_i, float(WINDOW)), 1.0)
430     else:
431         n_count = tl.maximum(n_i, 1.0)
432     temp = 1.0 + tl.load(SPG + h).to(tl.float32) * tl.log(n_count)
433     nullv = tl.load(NULLBASE + h).to(tl.float32) \
434         + tl.load(SPS + h).to(tl.float32) * tl.sqrt(tl.log(n_count + 1.0))
435     beta = tl.load(BETA + h).to(tl.float32)
436
437     # Online softmax state. Q2 is the extra accumulator needed for the
438     # participation ratio; it rescales by alpha**2 under max-shift updates.
439     m_i = tl.full([BLOCK_M], -1e38, tl.float32)
440     l_i = tl.zeros([BLOCK_M], tl.float32)
441     q2_i = tl.zeros([BLOCK_M], tl.float32)
442     acc = tl.zeros([BLOCK_M, DK], tl.float32)
443
444     for start_n in range(0, Tk, BLOCK_N):
445         offs_n = start_n + tl.arange(0, BLOCK_N)
446         k = load_key_block(K, offs_n) # (BLOCK_N, DK), native dtype
447         v = load_value_block(V, offs_n) # (BLOCK_N, DK), native dtype
448
449         sig = tl.dot(q, tl.trans(k)) * scale
450         a = sig * temp[:, None]
451         valid = offs_n[None, :] < n_i[:, None]
452         if WINDOW > 0:
453             valid = valid & (offs_n[None, :] >= (n_i[:, None] - WINDOW))
454         a = tl.where(valid, a, -1e38).to(tl.float32)
455
456         m_new = tl.maximum(m_i, tl.max(a, 1))
457         alpha = tl.exp(m_i - m_new)
458         p = tl.exp(a - m_new[:, None])
459         p = tl.where(valid, p, 0.0)
460
461         l_i = l_i * alpha + tl.sum(p, 1)
462         q2_i = q2_i * alpha * alpha + tl.sum(p * p, 1)
463         acc = acc * alpha[:, None] + tl.dot(p, v)
464         m_i = m_new
465
466     # Fold the virtual null sink as one additional softmax entry.
467     a_null = temp * nullv
468     m_new = tl.maximum(m_i, a_null)
469     alpha = tl.exp(m_i - m_new)
470     l_i = l_i * alpha
471     q2_i = q2_i * alpha * alpha
472     acc = acc * alpha[:, None]
473     m_i = m_new
474     p_null = tl.exp(a_null - m_i)
475     Z = l_i + p_null
476
477     v_null = tl.load(VNULL + h * DK + tl.arange(0, DK)).to(tl.float32)
478     s = acc + p_null[:, None] * v_null[None, :]
479     c = s / tl.maximum(tl.sqrt(tl.sum(s * s, 1)), eps)[:, None]
480
481     n_eff = l_i * l_i / tl.maximum(q2_i, eps)
482     m_eff = n_eff * (l_i / tl.maximum(Z, eps))
483     mag = 2.0 * tl.sigmoid(2.0 * beta * tl.log(1.0 + m_eff)) - 1.0
484
485     store_direction(c)
486     store_magnitude(mag)
487     save_for_backward(m_i, l_i, q2_i, s)
488

```

C Complete Ablation Sweep Results

We present the full numerical results of our 120-run factorial ablation sweep. The table lists the attention type, signature-stream regularization mode, distractor status, memory branch status, sliding-window status, training validation loss, clean-document perplexity and junk-stream perplexity at multipliers $1\times$ and $32\times$ (2K and 64K tokens), needle-in-a-haystack retrieval accuracy at 2K and 64K tokens, and training Model Flops Utilization (MFU).

Table 8: Complete Results of the 120-Run Ablation Sweep. All models are 370M parameters, trained on FineWeb-Edu for 1B tokens ($seq_len = 2048$). Evaluated from 2K to 64K context length. Clean PPL is measured on coherent FinePDFs documents; junk PPL is measured on the concatenated validation stream. PPL is in nats (lower is better), Needle is accuracy in % (higher is better), MFU is training model flops utilization in %.

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
POLAR	baseline	Off	Off	Off	3.323	2.83	3.60	3.27	5.68	96.3	0.0	40.1
POLAR	baseline	Off	Off	On	3.343	2.83	2.66	3.30	4.74	67.5	0.0	39.6
POLAR	baseline	Off	On	Off	3.169	2.70	1.96	3.11	3.13	91.3	92.5	35.6
POLAR	baseline	Off	On	On	3.174	2.71	2.01	3.12	3.13	51.3	30.0	35.6
POLAR	baseline	On	Off	Off	3.332	2.81	5.56	3.28	6.54	85.0	0.0	33.3
POLAR	baseline	On	Off	On	3.361	2.85	2.89	3.35	4.97	91.3	2.5	34.5
POLAR	baseline	On	On	Off	3.178	2.72	1.98	3.12	3.14	73.8	58.8	31.8
POLAR	baseline	On	On	On	3.182	2.73	1.97	3.12	3.14	56.3	21.3	31.2
POLAR	weak	Off	Off	Off	3.333	2.82	2.86	3.28	5.35	88.8	0.0	39.0
POLAR	weak	Off	Off	On	3.348	2.90	3.76	3.38	5.24	93.8	0.0	37.9
POLAR	weak	Off	On	Off	3.179	2.74	1.95	3.12	3.14	70.0	51.3	36.2
POLAR	weak	Off	On	On	3.187	2.74	1.99	3.13	3.15	41.3	41.3	36.2
POLAR	weak	On	Off	Off	3.335	2.82	3.47	3.28	5.56	91.3	0.0	32.8
POLAR	weak	On	Off	On	3.353	2.82	2.58	3.32	4.81	96.3	1.3	32.8
POLAR	weak	On	On	Off	3.178	2.74	1.95	3.12	3.14	33.8	32.5	30.7
POLAR	weak	On	On	On	3.183	2.74	1.94	3.13	3.15	81.3	78.8	30.7
POLAR	strong	Off	Off	Off	3.338	2.85	3.79	3.28	5.83	98.8	1.3	37.3

Continued on next page

Table 8: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
POLAR	strong	Off	Off	On	3.360	2.85	2.99	3.32	5.01	91.3	0.0	37.9
POLAR	strong	Off	On	Off	3.172	2.72	1.96	3.11	3.13	80.0	52.5	35.8
POLAR	strong	Off	On	On	3.181	2.72	1.94	3.12	3.15	76.3	68.8	34.6
POLAR	strong	On	Off	Off	3.337	2.87	3.22	3.28	5.52	83.8	1.3	32.8
POLAR	strong	On	Off	On	3.347	2.82	2.64	3.30	4.88	97.5	16.3	32.2
POLAR	strong	On	On	Off	3.179	2.73	1.94	3.12	3.14	86.3	83.8	30.9
POLAR	strong	On	On	On	3.182	2.72	1.94	3.12	3.14	25.0	23.8	30.9
POLAR	discrete	Off	Off	Off	3.338	2.82	2.80	3.29	5.12	93.8	2.5	37.9
POLAR	discrete	Off	Off	On	3.349	2.87	2.90	3.33	5.25	93.8	6.3	37.9
POLAR	discrete	Off	On	Off	3.184	2.75	2.03	3.12	3.14	76.3	73.8	36.4
POLAR	discrete	Off	On	On	3.178	2.71	1.92	3.12	3.15	31.3	18.8	35.8
POLAR	discrete	On	Off	Off	3.333	2.80	4.13	3.28	6.12	97.5	0.0	32.6
POLAR	discrete	On	Off	On	3.351	2.87	2.72	3.33	4.89	85.0	2.5	33.3
POLAR	discrete	On	On	Off	3.189	2.74	1.96	3.13	3.15	87.5	53.8	31.8
POLAR	discrete	On	On	On	3.192	2.74	2.07	3.13	3.15	21.3	26.3	31.7
POLAR	zipfian	Off	Off	Off	3.338	2.85	4.83	3.28	6.55	96.3	11.3	37.9
POLAR	zipfian	Off	Off	On	3.350	2.83	2.58	3.32	4.63	88.8	3.8	39.4
POLAR	zipfian	Off	On	Off	3.171	2.70	1.92	3.11	3.13	52.5	63.8	36.2
POLAR	zipfian	Off	On	On	3.180	2.75	1.95	3.12	3.15	46.3	40.0	35.2
POLAR	zipfian	On	Off	Off	3.340	2.83	3.77	3.28	5.64	98.8	1.3	33.8
POLAR	zipfian	On	Off	On	3.352	2.85	3.13	3.36	5.23	90.0	0.0	33.5
POLAR	zipfian	On	On	Off	3.177	2.72	1.94	3.12	3.14	91.3	67.5	31.8
POLAR	zipfian	On	On	On	3.192	2.74	2.00	3.13	3.15	40.0	26.3	31.8
NOPE	baseline	Off	Off	Off	3.224	2.76	3.71	3.16	6.68	97.5	1.3	41.5
NOPE	baseline	Off	Off	On	3.219	2.75	2.76	3.17	5.49	92.5	10.0	36.5
NOPE	baseline	Off	On	Off	3.140	2.66	2.34	3.09	3.23	97.5	16.3	36.8
NOPE	baseline	Off	On	On	3.150	2.70	2.14	3.09	3.24	81.3	0.0	34.2
NOPE	baseline	On	Off	Off	3.209	2.74	2.99	3.15	6.13	90.0	18.8	32.5
NOPE	baseline	On	Off	On	3.212	2.77	2.90	3.17	5.65	81.3	0.0	30.3
NOPE	baseline	On	On	Off	3.149	2.67	2.35	3.10	3.28	80.0	16.3	30.0

Continued on next page

Table 8: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
NOPE	baseline	On	On	On	3.147	2.68	2.09	3.09	3.23	88.8	21.3	28.2
NOPE	weak	Off	Off	Off	3.214	2.77	3.10	3.16	6.34	95.0	0.0	40.9
NOPE	weak	Off	Off	On	3.222	2.75	3.05	3.19	5.81	81.3	0.0	36.1
NOPE	weak	Off	On	Off	3.147	2.68	2.07	3.09	3.19	88.8	21.3	37.5
NOPE	weak	Off	On	On	3.142	2.68	2.12	3.08	3.23	82.5	0.0	33.7
NOPE	weak	On	Off	Off	3.215	2.75	4.87	3.16	12.48	95.0	5.0	32.0
NOPE	weak	On	Off	On	3.221	2.73	2.63	3.18	4.84	81.3	0.0	29.3
NOPE	weak	On	On	Off	3.147	2.67	2.48	3.09	3.23	97.5	1.3	30.8
NOPE	weak	On	On	On	3.148	2.68	2.14	3.09	3.24	93.8	1.3	28.4
NOPE	strong	Off	Off	Off	3.231	2.74	4.88	3.18	11.85	90.0	2.5	38.4
NOPE	strong	Off	Off	On	3.214	2.72	2.68	3.17	5.28	82.5	0.0	36.4
NOPE	strong	Off	On	Off	3.144	2.68	2.37	3.09	3.24	96.3	6.3	35.7
NOPE	strong	Off	On	On	3.150	2.70	2.10	3.10	3.25	93.8	7.5	33.8
NOPE	strong	On	Off	Off	3.207	2.72	3.13	3.15	6.20	73.8	0.0	31.4
NOPE	strong	On	Off	On	3.214	2.74	2.49	3.17	4.36	86.3	8.8	28.9
NOPE	strong	On	On	Off	3.146	2.67	2.24	3.09	3.23	83.8	17.5	29.1
NOPE	strong	On	On	On	3.147	2.68	2.16	3.09	3.28	96.3	0.0	27.5
NOPE	discrete	Off	Off	Off	3.202	2.75	3.33	3.15	6.23	85.0	1.3	38.8
NOPE	discrete	Off	Off	On	3.234	2.79	3.11	3.19	5.47	70.0	18.8	37.0
NOPE	discrete	Off	On	Off	3.145	2.68	2.14	3.09	3.27	98.8	16.3	36.0
NOPE	discrete	Off	On	On	3.145	2.68	2.10	3.09	3.25	95.0	3.8	34.3
NOPE	discrete	On	Off	Off	3.212	2.88	3.04	3.16	6.88	42.5	0.0	31.2
NOPE	discrete	On	Off	On	3.211	2.78	2.73	3.16	5.28	82.5	1.3	29.1
NOPE	discrete	On	On	Off	3.145	2.68	2.27	3.08	3.26	91.3	11.3	30.3
NOPE	discrete	On	On	On	3.149	2.68	2.17	3.08	3.32	83.8	0.0	28.6
NOPE	zipfian	Off	Off	Off	3.209	2.71	3.18	3.15	6.40	90.0	0.0	40.8
NOPE	zipfian	Off	Off	On	3.219	2.75	2.71	3.17	5.44	77.5	1.3	36.1
NOPE	zipfian	Off	On	Off	3.141	2.67	2.55	3.08	3.38	96.3	0.0	38.0
NOPE	zipfian	Off	On	On	3.150	2.69	2.11	3.09	3.25	90.0	3.8	33.7
NOPE	zipfian	On	Off	Off	3.207	2.77	3.32	3.15	7.09	82.5	0.0	31.3

Continued on next page

Table 8: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
NOPE	zipfian	On	Off	On	3.227	2.79	2.91	3.18	5.94	93.8	11.3	30.2
NOPE	zipfian	On	On	Off	3.146	2.68	2.39	3.10	3.25	96.3	21.3	29.6
NOPE	zipfian	On	On	On	3.148	2.68	2.17	3.09	3.28	90.0	6.3	28.7
ROPE	baseline	Off	Off	Off	3.159	2.85	3.36	3.09	5.21	42.5	0.0	40.4
ROPE	baseline	Off	Off	On	3.163	2.86	3.79	3.17	5.86	15.0	0.0	37.2
ROPE	baseline	Off	On	Off	3.145	2.81	2.86	3.09	3.57	73.8	0.0	37.1
ROPE	baseline	Off	On	On	3.151	2.80	2.95	3.12	3.72	13.8	0.0	34.4
ROPE	baseline	On	Off	Off	3.163	3.00	3.47	3.10	5.46	38.8	0.0	33.2
ROPE	baseline	On	Off	On	3.167	2.88	4.06	3.16	5.92	0.0	0.0	30.2
ROPE	baseline	On	On	Off	3.153	2.73	2.45	3.09	3.57	75.0	0.0	31.4
ROPE	baseline	On	On	On	3.154	2.83	2.86	3.12	3.72	32.5	0.0	29.4
ROPE	weak	Off	Off	Off	3.158	2.88	3.46	3.10	5.36	46.3	0.0	41.4
ROPE	weak	Off	Off	On	3.163	2.84	3.96	3.18	5.93	12.5	0.0	37.8
ROPE	weak	Off	On	Off	3.149	2.71	2.46	3.09	3.56	37.5	0.0	36.5
ROPE	weak	Off	On	On	3.156	2.85	2.77	3.12	3.60	23.8	0.0	34.9
ROPE	weak	On	Off	Off	3.155	2.83	3.49	3.10	5.27	38.8	0.0	32.3
ROPE	weak	On	Off	On	3.157	3.05	4.12	3.18	5.86	8.8	0.0	30.5
ROPE	weak	On	On	Off	3.149	2.77	2.76	3.09	3.56	57.5	0.0	31.1
ROPE	weak	On	On	On	3.149	2.77	3.13	3.12	3.80	17.5	0.0	28.0
ROPE	strong	Off	Off	Off	3.165	2.78	3.34	3.10	5.27	30.0	0.0	39.0
ROPE	strong	Off	Off	On	3.169	2.90	3.68	3.19	5.80	31.3	0.0	36.3
ROPE	strong	Off	On	Off	3.158	2.85	2.63	3.10	3.50	51.3	0.0	35.9
ROPE	strong	Off	On	On	3.162	2.86	3.08	3.14	3.76	10.0	0.0	33.2
ROPE	strong	On	Off	Off	3.161	2.85	3.58	3.10	5.28	47.5	0.0	31.1
ROPE	strong	On	Off	On	3.165	2.85	4.03	3.19	5.90	2.5	0.0	29.2
ROPE	strong	On	On	Off	3.155	2.80	2.70	3.10	3.62	61.3	0.0	29.3
ROPE	strong	On	On	On	3.162	2.85	3.08	3.13	3.74	2.5	0.0	28.4
ROPE	discrete	Off	Off	Off	3.165	2.91	3.43	3.11	5.42	22.5	0.0	39.7
ROPE	discrete	Off	Off	On	3.158	2.84	4.05	3.15	5.90	3.8	0.0	36.9
ROPE	discrete	Off	On	Off	3.159	2.83	2.67	3.11	3.56	47.5	0.0	36.3

Continued on next page

Table 8: Complete Results of the 120-Run Ablation Sweep (continued).

Type	Reg	Dist	Mem	Win	Val	Clean2K	Clean64K	Junk2K	Junk64K	Ndl2K	Ndl64K	MFU
ROPE	discrete	Off	On	On	3.161	2.79	2.71	3.13	3.74	41.3	0.0	33.7
ROPE	discrete	On	Off	Off	3.171	2.79	3.45	3.12	5.36	51.3	0.0	31.5
ROPE	discrete	On	Off	On	3.164	2.80	3.74	3.16	5.86	21.3	0.0	30.0
ROPE	discrete	On	On	Off	3.162	2.74	2.47	3.11	3.53	50.0	0.0	30.9
ROPE	discrete	On	On	On	3.154	2.79	2.68	3.11	3.67	18.8	0.0	28.9
ROPE	zipfian	Off	Off	Off	3.163	2.82	3.25	3.10	5.22	62.5	0.0	39.8
ROPE	zipfian	Off	Off	On	3.161	2.94	3.94	3.18	5.81	16.3	0.0	37.1
ROPE	zipfian	Off	On	Off	3.149	2.78	2.70	3.09	3.63	46.3	0.0	36.7
ROPE	zipfian	Off	On	On	3.151	2.80	2.63	3.11	3.68	2.5	0.0	33.9
ROPE	zipfian	On	Off	Off	3.161	2.78	3.38	3.10	5.32	47.5	0.0	32.6
ROPE	zipfian	On	Off	On	3.160	3.01	4.11	3.17	5.81	17.5	0.0	30.2
ROPE	zipfian	On	On	Off	3.149	2.73	2.39	3.09	3.55	62.5	0.0	29.8
ROPE	zipfian	On	On	On	3.152	2.79	2.99	3.12	3.78	26.3	0.0	28.5